

# FlareVM ARAÇ SETİ

[Kaynak](#)

## İÇERİK

1. FlareVM: Siber Güvenlik Araç Cephaneliği (Arsenal of Tools)
2. Soruşturma ve Analiz İçin Temel Araç Seti
3. Kötü Amaçlı Dosyaları İncelemek!

## FlareVM: Siber Güvenlik Araç Cephaneliği (Arsenal of Tools)

Siber güvenlik incelemelerinde, özellikle zararlı yazılım analizi ve adli bilişim vakalarında sistemin izole ve amaca yönelik araçlarla donatılmış olması kritik bir gereksinimdir. FlareVM, bu bağlamda adli bilişim (Forensics), olay müdahalesi (Incident Response) ve zararlı yazılım (Malware) incelemeleri için özelleştirilmiş, Windows tabanlı devasa bir cephaneliktir. Aşağıdaki notlar, bu sistem içerisinde yer alan araçların kategorik olarak teknik derinliğini ve analiz süreçlerindeki rollerini açıklamaktadır.

## Tersine Mühendislik ve Hata Ayıklama (Reverse Engineering & Debugging)

Tersine mühendislik işlemi, derlenmiş ve nihai haline ulaşmış bir kapalı kutu yazılımın (binary) mimarisini geriye doğru çözerek çalışma mantığını, algoritmalarını ve zafiyetlerini anlama sürecidir. Debugging (Hata ayıklama) ise bu kodun çalışma anındaki (runtime) davranışlarını adım adım izlemek, mantıksal hataları yakalamak ve bellek üzerindeki manipülasyonları tespit etmek için kullanılır.

Bu aşamada NSA tarafından geliştirilen **Ghidra**, karmaşık zararlı yazılımların kaynak koduna yakın pseudocode (sözde kod) çıktılarını elde etmek için kullandığımız en güçlü açık kaynaklı platformdur. Dinamik analiz sırasında, Windows işletim sistemlerindeki x64 ve x32 formatlı çalıştırılabilir dosyaların bellek adreslerinde ilerlemek için **x64dbg** kullanılırken, daha düşük seviyeli (Assembly düzeyinde) ve spesifik bellek analizleri için **OlllyDbg** tercih edilir. Komut satırı üzerinden çok daha esnek ve karmaşık operasyonlar yürütmek gerektiğinde ise gelişmiş bir platform olan **Radare2** devreye girer.

**ÖNEMLİ:** Herhangi bir analize başlamadan önce karşılaştığımız dosyanın sıkıştırılmış veya şifrelenmiş olup olmadığını bilmek zorundayız. Bu noktada **PEiD (Packer, Cryptor and Compiler Detection Tool)** kullanılır. Eğer dosya bir **Packer (Zararlı yazılımın asıl kodunu gizlemek ve analiz edilmesini zorlaştırmak için kullanılan paketleyici)** ile sarmalanmışsa, decompile işlemi başarısız olur veya anlamsız veriler üretir. Bu yüzden ilk adım daima PEiD veya benzeri bir araçla dosyanın imzasını doğrulamaktır. İkili dosyaları

parçalamak ve yapısal bir analiz sunmak içinse **Binary Ninja** etkili bir disassembler olarak kullanılır.

## Disassembler ve Decompiler Çözümleri

Bir zararlı yazılım analisti için makine kodunu okumak neredeyse imkansızdır. Disassembler ve Decompiler araçları, bu makine kodunu alarak kontrol akışını (control flow), çalışma mantığını (logic) ve davranışsal modellerini insan tarafından okunabilir Assembly veya yüksek seviyeli dillere (C/C++ gibi) çevirir.

Windows tabanlı bir analizin kalbinde **CFF Explorer** yatar. Bu araç, **PE (Portable Executable - Windows işletim sistemlerinin kullandığı çalıştırılabilir dosya formatı)** dosyalarını düzenlemek ve analiz etmek için kullanılır. Özellikle zararlıının hangi sistem kütüphanelerini çağırdığını (Import Address Table - IAT) inceleyerek yazılımın ne yapmaya çalıştığını (örneğin; ağa mı bağlanacak, dosyaları mı şifreleyecek?) henüz çalıştırmadan tahmin edebiliriz. **Hopper Disassembler**; debugger, disassembler ve decompiler özelliklerini tek bir çatıda sunan hibrit bir araçken, makine kodunu doğrudan yüksek seviyeli bir dile çevirmek istediğimizde açık kaynaklı **RetDec** motorunu kullanırız.

## Statik ve Dinamik Analiz (Static & Dynamic Analysis)

Zararlı yazılım analizi metodolojisi iki temel sütuna dayanır. Statik Analiz, dosyayı kesinlikle çalıştırmadan (execute etmeden) sadece kod bloklarını, metin dizilerini (strings) ve dosya başlıklarını incelemektir. Dinamik Analiz ise zararlıyı izole bir ortamda bilerek çalıştırıp sisteme, ağa ve belleğe olan etkilerini gerçek zamanlı izleme sürecidir.

Sistemde çalışan süreçleri analiz etmek için standart görev yöneticisi yetersiz kalır; bu yüzden bellek düzenleme ve gelişmiş süreç izleme yeteneklerine sahip **Process Hacker** kullanılır. PE başlıklarını detaylıca incelemek için **PEview** başvurulana ana araçtır.

Eğer bir yazılımın sistemde doğru çalışıp çalışmayacağını veya hangi kütüphaneleri sömüreceğini anlamak istiyorsak **Dependency Walker** kullanılır. Bu araç, çalıştırılabilir dosyanın bağımlı olduğu **DLL (Dynamic Link Library - Dinamik Bağlantı Kütüphanesi)** dosyalarını hiyerarşik bir ağaç yapısında gösterir. Statik analizin tespit yeteneklerini güçlendirmek için PEiD'nin daha modern bir muadili olan **DIE (Detect It Easy)** ile paketleyici ve derleyici tespiti yapılır.

## Adli Bilişim ve Olay Müdahalesi (Forensics & Incident Response)

Adli Bilişim (Digital Forensics); bilgisayarlar, ağlar ve depolama aygıtlarındaki dijital delillerin bütünlüğü bozulmadan toplanması, analiz edilmesi ve korunması sürecidir. Olay Müdahalesi (Incident Response) ise bir siber saldırı gerçekleştiği anda tespiti yapmak, saldırıyı sınırlandırmak (containment), tehdidi sistemden kazımak (eradication) ve iş sürekliliğini sağlamak odaklıdır.

Özellikle "Fileless Malware" (Dosyasız zararlı yazılım) olarak adlandırılan ve diske hiçbir veri yazmadan doğrudan bellekte (RAM) çalışan tehditleri yakalamak için bellek analizi şarttır. Bu noktada **Volatility** ve **Rekall**, **RAM** imajlarını incelemek için kullanılan iki devasa framework'tür. Disk üzerinde bir inceleme veya imaj alma süreci başlatılacaksa, kriptografik doğruluğu sağlamak adına endüstri standardı olan **FTK Imager** kullanılır.

Aşağıda Volatility aracının bir bellek dökümündeki ağ bağlantılarını bulmak için çalıştırılma mantığına örnek verilmiştir:

```
volatility -f infected_memory.vmem --profile=Win7SP1x64 netscan
```

*Açıklama: -f parametresiyle analiz edilecek RAM imajı (infected\_memory.vmem) belirtilir. -profile parametresi son derece kritiktir; Volatility'nin bellek adreslerini doğru haritalandırabilmesi için imajı alınan sistemin tam işletim sistemi versiyonunu (Win7 SP1 x64) bilmesi gerekir. netscan eklentisi (plugin) ise zararlı yazılımın enfekte olduğu anda açık tuttuğu veya sonlandırdığı tüm TCP/UDP ağ bağlantılarını dökerek C2 (Command & Control) sunucusu IP'sini tespit etmemizi sağlar. Bu adım başarısız olursa, alternatif olarak connscan eklentisine başvurulmalıdır.*

## Ağ Analizi (Network Analysis)

Sistemdeki zararlı aktiviteyi tespit etmenin en kesin yollarından biri ağ trafiğini dinlemektir. Ağ analizi, trafiğin optimizasyonunun yanı sıra anormalliklerin, veri sızıntılarının ve yetkisiz erişimlerin tespiti için kullanılır.

Trafiği byte seviyesine inerek okumak ve paketlerin (PCAP) içeriğini incelemek için **Wireshark** kullanılır. Hedef ağdaki zafiyetleri ve açık portları haritalandırmak için sektörün vazgeçilmezi **Nmap**'tir.

**Netcat** ise TCP/IP'nin İsviçre çakısıdır; ağ bağlantıları üzerinden doğrudan veri okumak ve yazmak için kullanılır. Özellikle saldırgan perspektifinden bir **Reverse Shell (Hedef sistemin, komut satırı kontrolünü ağ üzerinden saldırganın dinlediği porta kendi isteğiyle göndermesi)** bağlantısı kurmak veya yakalamak için en temel araçtır.

## Dosya Analizi (File Analysis)

Potansiyel güvenlik tehditlerini anlamak için dosyaların en ilkel haline, yani Hexadecimal (Onaltılık) byte formatına inilmesi gerekir. Bu işlem, standart metin editörleriyle okunamayan çalıştırılabilir (binary) dosyaların iç yapısını manipüle etmeyi sağlar.

**FileInsight**, binary dosyalarını düzenlemek ve kod blokları arasında arama yapmak için kullanılırken; **Hex Fiend** hafif ve hızlı bir alternatif sunar. **HxD** ise yüksek boyutlu dosyaları dahi donmadan açabilen, doğrudan RAM belleğine müdahale edebilen son derece güçlü bir hex editörü ve görüntüleyicisidir. Bu araçlarla dosya içindeki "Magic Bytes" (bir dosyanın formatını belirten ilk birkaç byte) değiştirilerek zararlının dosya uzantısı manipülasyonları tespit edilebilir.

## Betik Yazma ve Otomasyon (Scripting & Automation)

Tekrarlayan analiz görevlerini manuel yapmak zaman kaybıdır ve insan hatasına (human error) açıktır. Özellikle yüzlerce dosyanın statik analizinin yapıldığı senaryolarda otomasyon hayati bir gereksinimdir.

Bunun için **Python**, içerdiği modüller ve esnekliği ile analiz araçlarını birbirine bağlayan temel dildir. Microsoft ortamında ise sızma sonrası (post-exploitation) işlemler için **PowerShell Empire** kullanılır. Empire, **PowerShell**'in gücünü kullanarak hedef sistemde kalıcılık sağlama, kimlik bilgisi hırsızlığı yapma ve sistem yöneticisi yetkilerini sömürme gibi işlemleri otomatize eden bir framework'tür.

## Sysinternals Paketi (Sysinternals Suite)

Microsoft tarafından geliştirilen Sysinternals, bir IT profesyoneli ve güvenlik analisti için Windows sistemlerde derinlemesine sorun giderme (troubleshooting) ve zararlı davranış tespiti yapmanın temel taşıdır.

Sistem boot (başlatılma) aşamasında hangi dosyaların ve registry (kayıt defteri) anahtarlarının otomatik olarak tetiklendiğini görmek için **Autoruns** kullanılır. Zararlı yazılımlar genellikle sistem her yeniden başladığında hayatta kalmak için (Kalıcılık/Persistence) kendilerini bu başlangıç yollarına eklerler.

**Process Explorer**, görev yöneticisinin aksine sistemdeki tüm parent-child (ebeveyn-çocuk) süreç ilişkilerini göstererek zararlıının hangi meşru sürecin altına gizlendiğini (Process Injection) ortaya çıkarır.

**DİKKAT: Process Monitor (ProcMon)** aracı, analiz sürecinin göz bebeğidir. Sistemdeki tüm dosya, registry ve thread (iş parçacığı) aktivitelerini gerçek zamanlı (real-time) olarak yakalar ve loglar. Bir zararlı yazılım çalıştırıldığında; diskin neresine hangi dosyayı yazdığını (CreateFile), registry'de hangi güvenlik ayarını değiştirdiğini (RegSetValue) eksiksiz görmek için ProcMon logları incelenmelidir. Sistemin kalbine giden yol bu kayıtları doğru filtrelemekten geçer.

## Soruşturma ve Analiz İçin Temel Araç Seti (Commonly Used Tools for Investigation)

Siber güvenlik olay müdahalesi (Incident Response) ve zararlı yazılım analizinin (Malware Analysis) ilk aşaması, sistemde ne olduğunu anlamak için doğru araçları doğru bağlamda kullanmaktır. Bu aşamada (Triage), süreci körü körüne yürütmek yerine, dosya sistemini, belleği, ağ trafiğini ve çalıştırılabilir dosyaların (PE - Portable Executable) iç yapılarını derinlemesine incelememize olanak tanıyan özel araçlara ihtiyaç duyarız.

Aşağıda, bir güvenlik analistinin cephaneliğinde bulunması gereken temel araçların teknik derinliği ve analiz sırasındaki mantıksal kullanım senaryoları incelenmiştir.

## Process Monitor (Procmon)

İşletim sistemi seviyesindeki her türlü dosya, kayıt defteri (Registry) ve iş parçacığı/süreç (Thread/Process) aktivitesini gerçek zamanlı (real-time) olarak izlememizi sağlayan, Microsoft Sysinternals paketinin en kritik araçlarından biridir. Zararlı yazılım analizinde bir programın diskin neresine dokunduğunu, sistem başlangıcına kendini ekleyip eklemediğini (Persistence) ve hangi sistem yapılandırmalarını değiştirdiğini anlamak için Procmon logları incelenir.



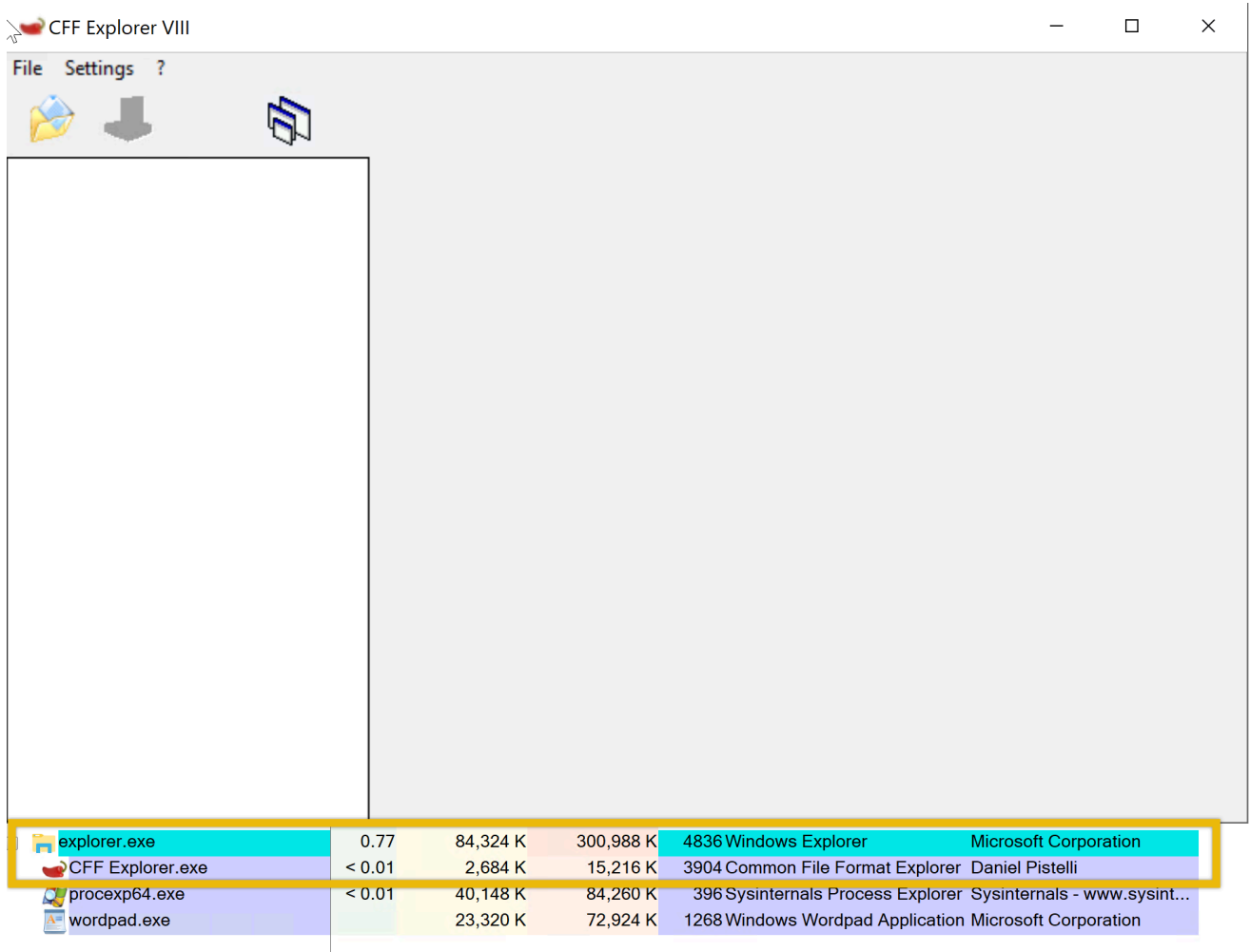
| Time of Day         | Process   | PID | Operation | Path                           | Result  | Detail                                                                         |
|---------------------|-----------|-----|-----------|--------------------------------|---------|--------------------------------------------------------------------------------|
| 10:35:01.0820760 PM | lsass.exe | 804 | ReadFile  | C:\Windows\System32\samsrv.dll | SUCCESS | Offset: 867,328, Length: 13,312, I/O Flags: Non-cached, Paging I/O, Synchron   |
| 10:35:01.0834147 PM | lsass.exe | 804 | ReadFile  | C:\Windows\System32\samsrv.dll | SUCCESS | Offset: 855,040, Length: 12,288, I/O Flags: Non-cached, Paging I/O, Synchron   |
| 10:35:01.0834733 PM | lsass.exe | 804 | ReadFile  | C:\Windows\System32\lsasrv.dll | SUCCESS | Offset: 1,621,504, Length: 13,312, I/O Flags: Non-cached, Paging I/O, Synchron |
| 10:35:01.0843959 PM | lsass.exe | 804 | ReadFile  | C:\Windows\System32\lsasrv.dll | SUCCESS | Offset: 1,605,120, Length: 16,384, I/O Flags: Non-cached, Paging I/O, Synchron |
| 10:35:01.0844286 PM | lsass.exe | 804 | ReadFile  | C:\Windows\System32\samsrv.dll | SUCCESS | Offset: 794,112, Length: 16,384, I/O Flags: Non-cached, Paging I/O, Synchron   |

Analiz sırasında karşılaşılan spesifik bir senaryo üzerinden gidelim: Procmon loglarında, Windows'un temel güvenlik ve kimlik doğrulama süreçlerini yöneten `lsass.exe` (Local Security Authority Subsystem Service) sürecinin bir dosyayı başarıyla okuduğuna dair bir girdi görebiliriz. `lsass.exe`, yapısı gereği `lsasrv.dll` (Local Security Authority Server Service) gibi kritik kütüphanelerle sürekli iletişim halindedir ve bu normal bir davranıştır.

**ÖNEMLİ:** `lsass.exe`'nin kendisi zararlı değildir; ancak saldırganların bir numaralı hedeflerinden biridir. Mimikatz gibi araçlar kullanılarak gerçekleştirilen **Credential Dumping (Kimlik Bilgisi Hırsızlığı)** saldırılarında, saldırganlar LSASS belleğine (RAM üzerindeki alanına) enjekte olarak sistemdeki açık metin parolaları, NTLM hash'lerini veya Kerberos biletlerini çalmaya çalışırlar. Bu nedenle Procmon üzerinde analiz yaparken, `lsass.exe`'nin standart dışı bir süreç (Örn: geçici bir klasörden çalışan rastgele isimlendirilmiş bir .exe) tarafından okunmaya (Read) veya yazılmaya (Write) çalışıldığını, ya da bellek alanına şüpheli bir erişim (Handle isteği) yapıldığını görürseniz, bu doğrudan bir yetki yükseltme veya yanal hareket hazırlığına işaret eder.

## Process Explorer (Procexp)

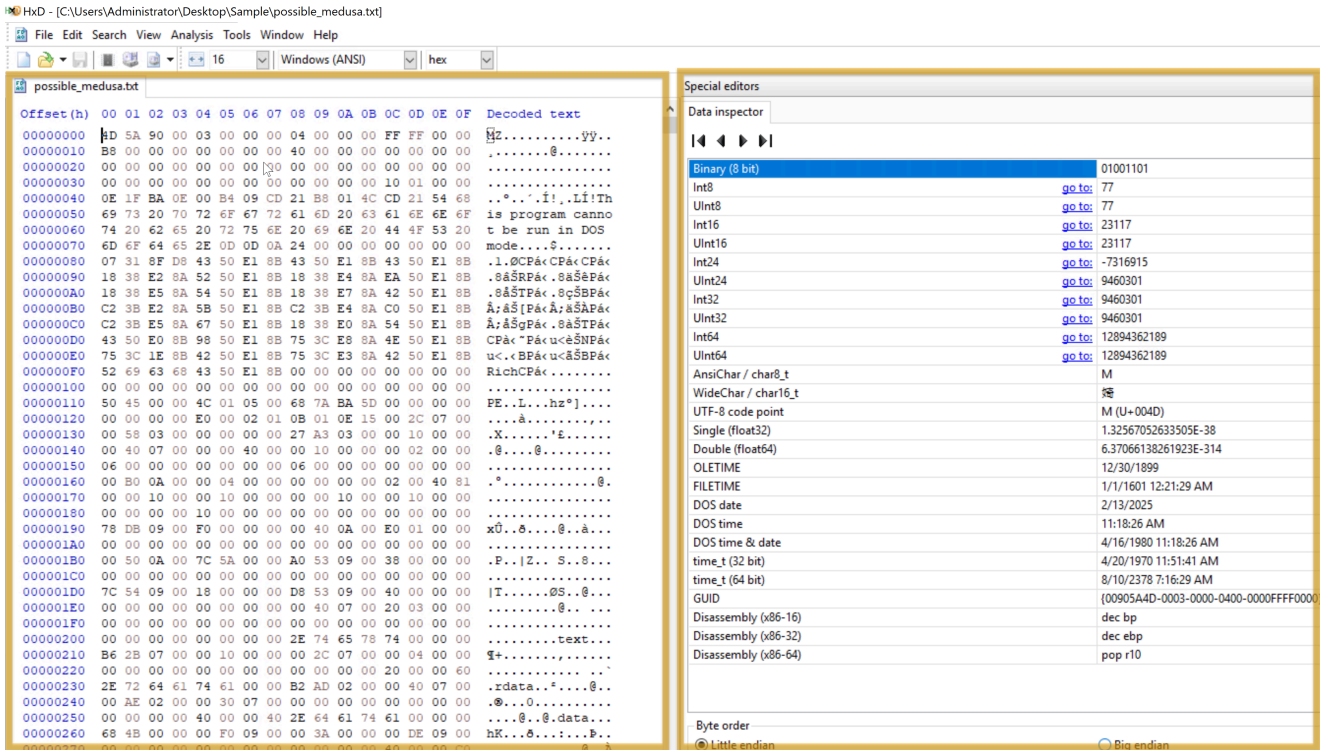
Görev Yöneticisi'nin (Task Manager) çok daha gelişmiş bir versiyonudur. Sistemdeki süreçlerin yalnızca CPU veya RAM tüketimini değil; **Parent-Child (Ebeveyn-Çocuk) ilişkilerini**, sürecin belleğine yüklenmiş **DLL'eri (Dynamic Link Libraries)** ve sürecin disk üzerindeki tam yolunu hiyerarşik bir ağaç yapısında gösterir.



Neden Parent-Child ilişkisini izliyoruz? Çünkü zararlı yazılım tespitinde bağlam (context) her şeydir. Örneğin, bir zararlı yazılım analiz ortamında CFF Explorer uygulamasını açtığınızda, Process Explorer üzerinde `explorer.exe` 'nin (Masaüstü ortamı) altında çalıştığını görürsünüz. Bu meşru bir davranıştır. Ancak, eğer bir Microsoft Word belgesi ( `winword.exe` ), altında bir komut satırı ( `cmd.exe` ) veya PowerShell ( `powershell.exe` ) başlatıyorsa (spawn ediyorsa), bu %99 oranında bir Oltalama (Phishing) veya Makro tabanlı bir ilk erişim (Initial Access) saldırısıdır. Tehdit aktörleri genellikle zararlı kodlarını meşru görünen .LNK (Kısayol) dosyaları veya .ISO imajları üzerinden çalıştırarak, arka planda işletim sisteminin yasal süreçlerini (Living off the Land) istismar ederler. Process Explorer, bu anormallikleri saniyeler içinde görselleştirmemizi sağlar.

## HxD (Gelişmiş Hex Editörü)

Bir dosyanın işletim sistemi tarafından nasıl yorumlandığına güvenemeyiz. Saldırganlar, çalıştırılabilir bir zararlı yazılımın uzantısını .txt veya .jpg olarak değiştirerek (Extension Spoofing) analiz araçlarını ve kullanıcıları atlatmaya çalışırlar. HxD, dosyaları diskteki en ham haliyle, yani Onaltılık (Hexadecimal) veri formatında incelememize ve manipüle etmemize olanak tanır.



Bir adli biliřim incelemesinde `possible_medusa.txt` isimli řüpheli bir metin dosyası bulunduğumuzu varsayalım. Dosyayı standart bir metin editöründe açmak yerine HxD ile açtığımızda sol tarafta hex değerlerini, sağ tarafta ise ASCII (metin) karşılığını görürüz.

**DİKKAT:** Eğer bir dosya hex görünümünde `4D 5A` (ASCII karşılığı **MZ**) baytları ile başlıyorsa, uzantısı ne olursa olsun o dosya bir Windows çalıştırılabilir dosyasıdır (PE - Portable Executable). Bu `4D 5A` değerine **Magic Bytes (Dosya İmzası)** denir ve dosyanın başlığında (Header) yer alır. HxD'nin Data Inspector özelliğı, seçilen baytların Integer, Float veya Little Endian formatındaki değerlerini anında çevirerek, zararlı yazılımın içine gömülmüş olası IP adreslerini, şifreleme anahtarlarını veya komuta kontrol (C2) sunucusu URL'lerini tespit etmemizi sağlar.

## CFF Explorer

Çalıştırılabilir dosyaların (PE) başlık bilgilerini, yüklediğı kütüphaneleri ve bölüm (Section) detaylarını analiz etmek için kullanılan vazgeçilmez bir araçtır. Ayrıca dosyaların bütünlüğünü (Integrity) doğrulamak için kriptografik özetlerini (Hash) hesaplar.



File Settings ?

File: cryptominer.bin

- Dos Header
- Nt Headers
  - File Header
  - Optional Header
  - Data Directories [x]
- Section Headers [x]
- Resource Directory
- .NET Directory
  - MetaData Header
  - MetaData Streams
    - #~
      - Tables Header
      - Tables
    - #Strings
    - #US
    - #GUID
    - #Blob

Address Converter  
Dependency Walker  
Hex Editor  
Identifier  
Import Adder  
Quick Disassembler  
Rebuilder  
Resource Editor

| Property  | Value                                                 |
|-----------|-------------------------------------------------------|
| File Name | C:\Users\Administrator\Desktop\Sample\cryptominer.bin |
| File Type | Portable Executable 64 .NET Assembly                  |
| File Info | No match found.                                       |
| File Size | 1.91 MB (2004480 bytes)                               |
| PE Size   | 1.91 MB (2004480 bytes)                               |
| Created   | Monday 23 September 2024, 20.43.53                    |
| Modified  | Tuesday 24 September 2024, 03.43.38                   |
| Accessed  | Monday 23 September 2024, 20.43.56                    |
| MD5       | 9FDD4767DE5AEC8E577C1916ECC3E1D6                      |
| SHA-1     | A1BC55A7931BFCD24651357829C460FD3DC4828F              |

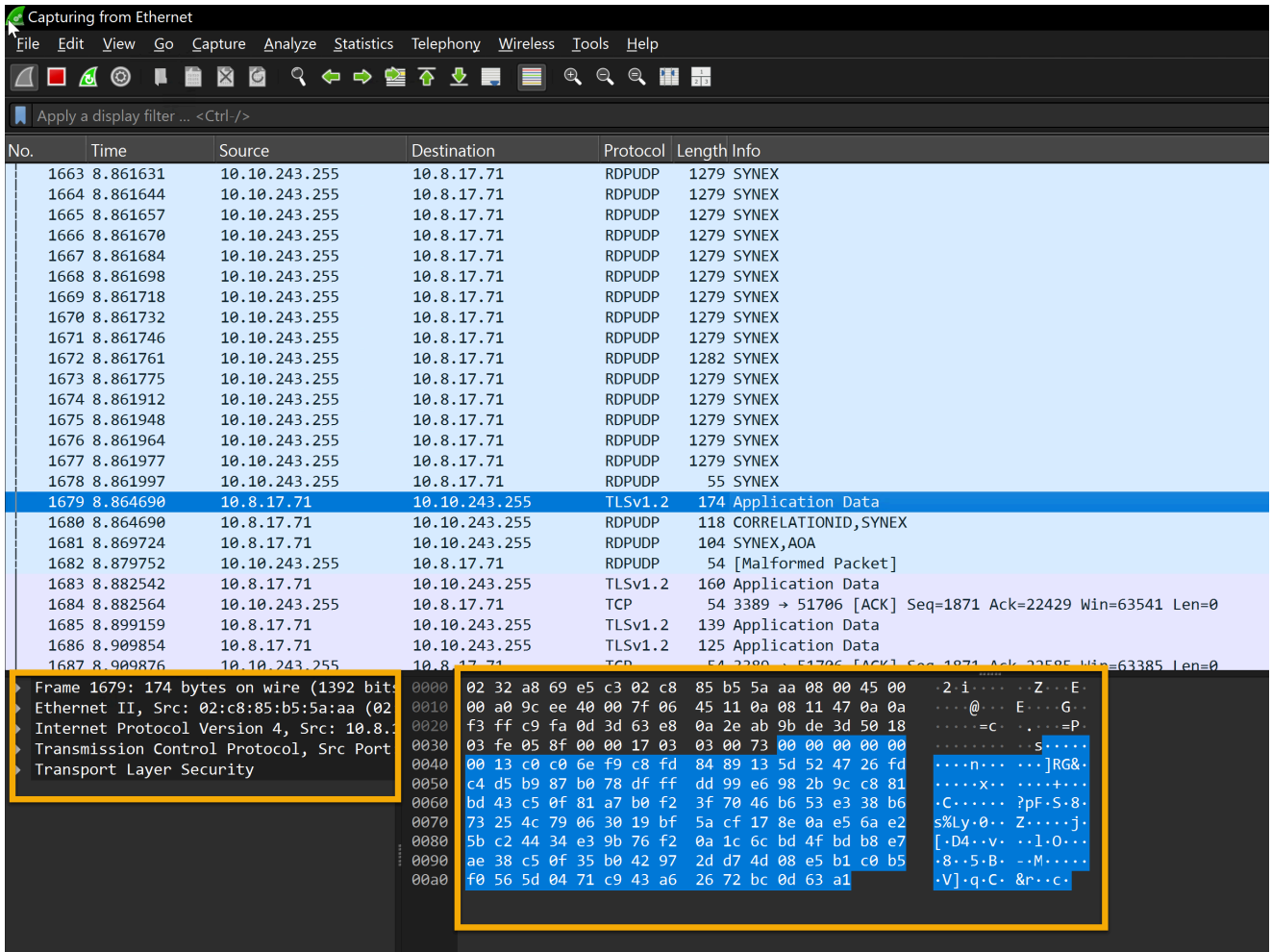
| Property         | Value                                        |
|------------------|----------------------------------------------|
| Comments         | Редактор реестра                             |
| CompanyName      | Операционная система Microsoft® Windows®     |
| FileDescription  | REGEDIT                                      |
| FileVersion      | 6.3.9600.16384 (winblue_rtm                  |
| InternalName     | etc-50-100=new.exe                           |
| LegalCopyright   | © Корпорация Майкрософт. Все права защищены. |
| OriginalFilename | etc-50-100=new.exe                           |
| ProductName      | Microsoft Corporation                        |
| ProductVersion   | 6.3.9600.16384 (winblue_rtm                  |

Örneğin, cryptominer.bin adlı şüpheli bir dosyanın CFF Explorer ile analiz edildiğinde, bunun aslında 64-bit bir PE (çalıştırılabilir) dosya olduğu ve 23 Eylül 2024'te derlendiği bilgisini (Compile Time) görebiliriz. Araç aynı zamanda dosyanın SHA-1 ve MD5 hash değerlerini üretir. Bu hash değerleri, dosyanın bilinen bir zararlı olup olmadığını VirusTotal gibi tehdit istihbaratı platformlarında sorgulamak için kullanılır. Ayrıca, işletim sisteminin kritik bir bileşeninin (örneğin svchost.exe) değiştirilip değiştirilmediğini (File Replacement Attack), orijinal dosyanın hash'i ile elimizdeki dosyanın hash'ini karşılaştırarak doğrulayabiliriz.

## Wireshark

Dosya tabanlı analizleri bitirdikten sonra ağ tabanlı (Network-based) analizlere geçeriz. Wireshark, ağ arayüzünden geçen paketleri yakalayan ve protokol (TCP, UDP, HTTP, DNS vb.) seviyesinde derinlemesine inceleme imkanı sunan bir paket analizörüdür (Packet Sniffer). Şüpheli bağlantıları, veri sızdırmalarını (Data Exfiltration) veya sisteme sızma girişimlerini tespit etmek için kullanılır.



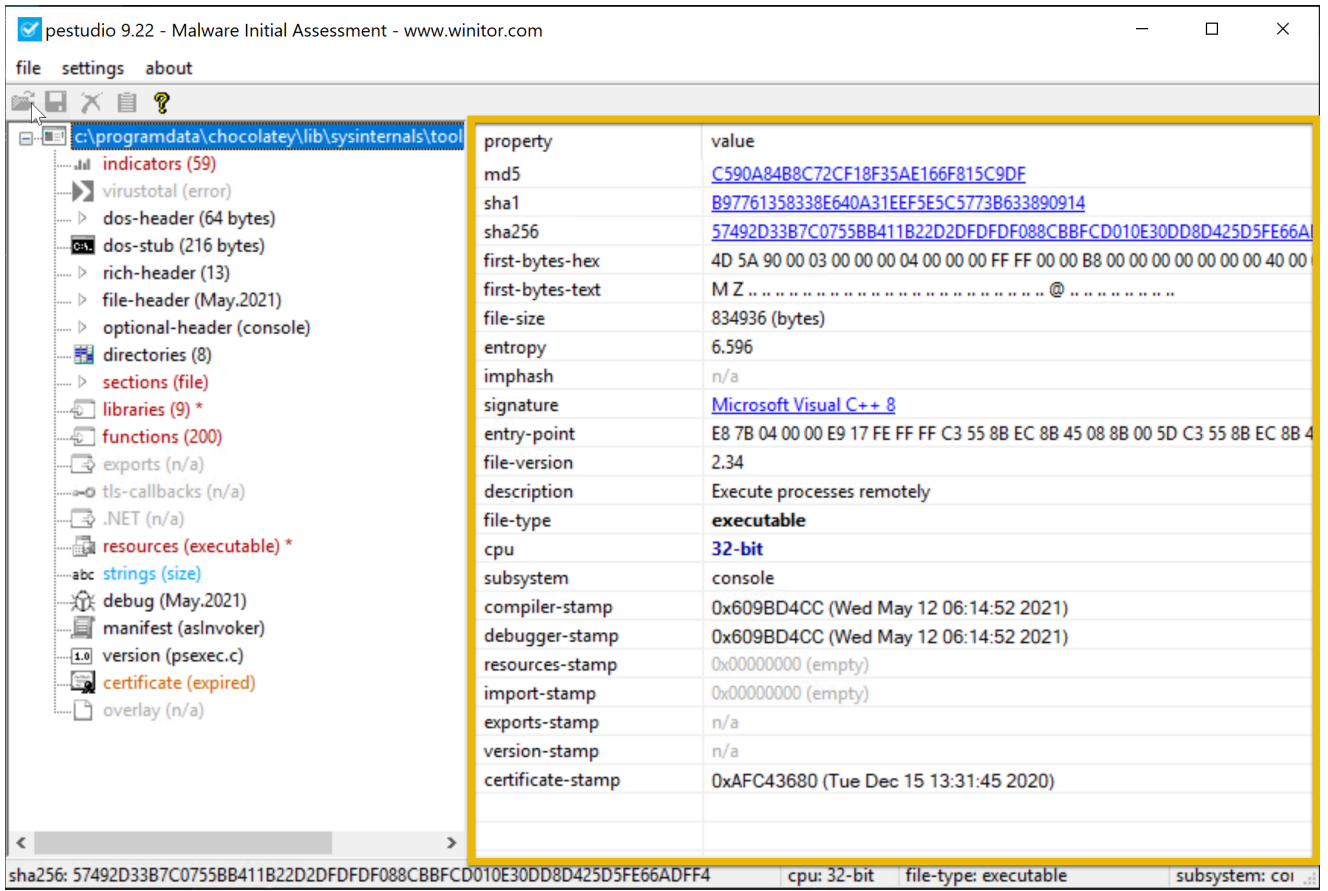


Ağ trafiğini incelerken, dışarıya giden bağlantıların çoğunda **TLSv1.2 (Transport Layer Security)** ve **TCP** protokollerinin kullanıldığını görebiliriz. TLS, veriyi şifrelediği için paketin içeriğini (payload) doğrudan ASCII veya Hex olarak okuyamayız.

**ÖNEMLİ:** Zararlı yazılımlar, güvenlik duvarlarını ve IDS/IPS sistemlerini atlatmak için kendi C2 (Komuta Kontrol) sunucularıyla haberleşirken özellikle TLS kullanırlar. Trafik şifreli olsa bile Wireshark üzerinden hedef IP adresi, kullanılan portlar, bağlantı sıklığı (Beaconing) ve TLS el sıkışmasındaki (Handshake) sertifika bilgileri (SNI - Server Name Indication) incelenerek trafiğin meşru bir bulut servisine mi yoksa şüpheli bir sunucuya mı gittiği tespit edilebilir.

## PEStudio

**Static Analysis (Statik Analiz)**, bir dosyayı asla çalıştırmadan, içindeki potansiyel tehlikeleri ve yapısal özellikleri ortaya çıkarma sanatıdır. PEStudio, bu işi otomatize eden ve analisti riskten tamamen yalıtın bir araçtır. Bir PE dosyasını PEStudio'ya yüklediğimizde bize dosyanın ne kadar şüpheli olduğuna dair zengin bir metrik seti sunar.



Örneğin, bir sistemde PSEXEC.exe dosyası yakaladığımızı düşünelim. PEStudio çıktısında ilk bakacağımız yer **Entropy (Entropi)** değeridir. Entropi, bir dosyanın içindeki verinin ne kadar karmaşık/rastgele olduğunu 0 ile 8 arasında bir değerle ölçer. Çıktıda Entropi değerinin **6.596** olduğunu görüyoruz. *Neden önemli?* Yüksek entropi (genellikle 7.0 ve üzeri), dosyanın bir **Packer (Sıkıştırıcı)** ile paketlenmişine veya içindeki asıl zararlı kodun şifrelenmişine (Encrypted) güçlü bir işarettir. 6.596 değeri, orta-yüksek bir değer olup, dosyanın şüpheli özellikler barındırdığını gösterir.

Ayrıca PEStudio, bunun Visual C++ 8 ile derlenmiş 32-bit (v2.34) bir konsol uygulaması olduğunu belirtir. PsExec, normalde sistem yöneticileri tarafından uzak makinelerde komut çalıştırmak için kullanılan yasal bir Microsoft Sysinternals aracıdır. Ancak siber saldırganlar, sisteme ilk sızmayı gerçekleştirdikten sonra ağ içindeki diğer cihazlara yayılmak, yani **Lateral Movement (Yatay Hareket)** yapmak ve uzaktan kod çalıştırmak (RCE - Remote Code Execution) için PsExec'i (veya benzeri araçları) sıklıkla istismar ederler. Yasal bir aracın, yetkisiz bir ortamda, ortalama bir entropi ile bulunması (Post-Exploitation phase), sistemde kesin bir ihlal (Compromise) olduğuna işaret eder.

## FLOSS (FLARE Obfuscated String Solver)

Statik analizin en temel adımı, derlenmiş dosyanın içindeki okunabilir metinleri (Strings) çıkartmaktır. Klasik strings.exe aracı sadece dosyanın içindeki düz metinleri bulabilir. Ancak FLOSS (eski adıyla FireEye Labs Obfuscated String Solver), gelişmiş statik analiz teknikleri ve emülasyon kullanarak zararlı yazılım yazarlarının özellikle gizlediği veya şifrelediği (Obfuscated) metin dizilerini otomatik olarak çözüp (De-obfuscate) ortaya çıkarır.

Aşağıdaki komut ve log çıktısını detaylıca inceleyelim:

## PowerShell

```
# FLOSS aracını şüpheli cobaltstrike.exe dosyasına parametresiz (varsayılan ayarlarla) çalıştırıyoruz.
PS C:\Users\Administrator\Desktop\Sample > floss .\cobaltstrike.exe

INFO: floss: extracting static strings
# Araç önce standart ASCII ve UTF-16 dizilerini çekiyor.
INFO: floss.stackstrings: extracting stackstrings from 50 functions
# Stack Strings: Zararlı yazılımlar dizileri tek bir blok halinde tutmak yerine, çalışma anında bellekte (Stack) karakter karakter birleştirir. FLOSS bu 50 fonksiyonu analiz ediyor.
INFO: floss.tightstrings: extracting tightstrings from 4 functions...
INFO: floss.string_decoder: decoding strings
# FLOSS, şifreli olabileceğini düşündüğü 1 fonksiyonu (0x402e80) emüle ederek çalıştırıyor (sanal olarak) ve çözmeye çalışıyor.
INFO: floss: finished execution after 265.61 seconds
```

İşlem bittikten sonra elde edilen nihai rapor (Results):

| FLARE FLOSS RESULTS (version v3.1.0-0-gdb9af41) |                       |
|-------------------------------------------------|-----------------------|
| +-----+-----                                    |                       |
| file path                                       | cobaltstrike.exe      |
| identified language                             | unknown               |
| extracted strings                               |                       |
| static strings                                  | 189 (2050 characters) |
| language strings                                | 0 ( 0 characters)     |
| stack strings                                   | 0                     |
| tight strings                                   | 0                     |
| decoded strings                                 | 0                     |
| +-----+-----                                    |                       |

Raporun devamında, elde edilen **189 Statik String**'in dökümü yer almaktadır. Bu dökümdeki kritik değerlerin teknik anlamları şunlardır:

```
+-----+
| FLOSS STATIC STRINGS: ASCII (188) |
+-----+

!This program cannot be run in DOS mode.
.text
P`.data
.rdata
```

*Yorum:* Dosyanın kesinlikle bir PE (Windows Çalıştırılabilir) dosyası olduğunu ve standart PE bölümlerini (sections) içerdiğini doğruluyoruz.

```
CloseHandle
ConnectNamedPipe
CreateFileA
CreateNamedPipeA
CreateThread
VirtualAlloc
VirtualProtect
WriteFile
KERNEL32.dll
```

*Yorum:* Çıkarılan bu Windows API (Application Programming Interface) çağrıları dosyanın davranış profili hakkında %100 net bir fikir verir. `VirtualAlloc` (bellekte alan ayırma), `WriteFile` (veri yazma), `VirtualProtect` (bellek alanının izinlerini değiştirme, örn: çalıştırılabilir yapma) ve `CreateThread` (iş parçacığı başlatma) API'lerinin bir arada bulunması, **Process Injection (Süreç Enjeksiyonu)** veya **Shellcode Execution** yapıldığının ders kitaplarındaki karşılığıdır. `ConnectNamedPipe` ve `CreateNamedPipeA` ise, zararlının C2 (Command & Control) ile haberleşirken veya yerel sistemdeki diğer zararlı süreçlerle iletişim kurarken **Named Pipes** (İsmlendirilmiş Borular - genellikle Cobalt Strike tarafından sıkça kullanılır) kullandığını gösterir.

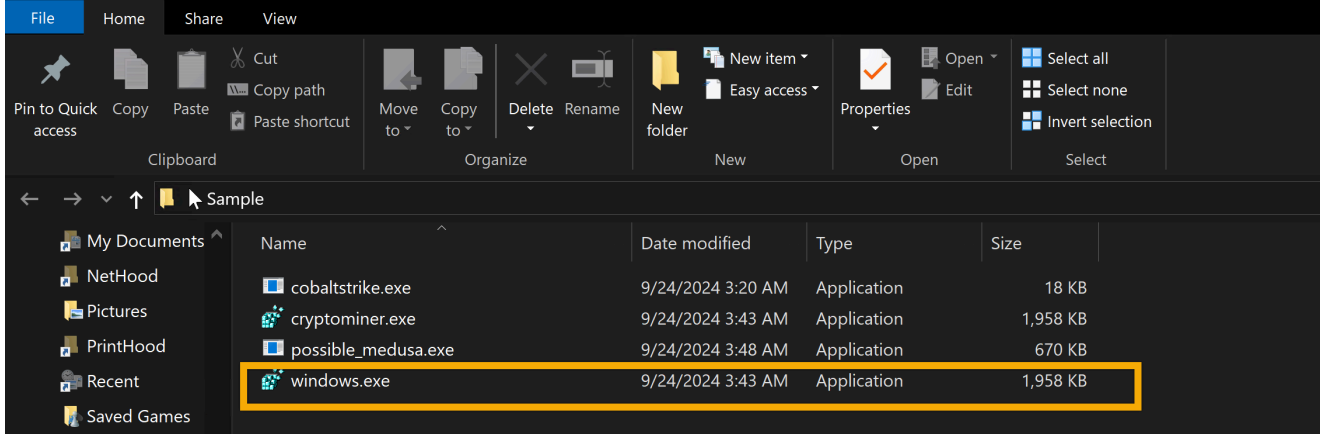
**ÖNEMLİ ANALİZ SONUCU:** FLOSS raporunda `stack strings`, `tight strings` ve `decoded strings` değerlerinin "0" olduğunu görüyoruz. Bu, FLOSS'un emülasyon yeteneklerinin dinamik olarak şifrelenmiş veya gizlenmiş (obfuscated) hiçbir string bulamadığını veya çözemediğini gösterir. Bu durum iki anlama gelir: Ya dosya yazarı gelişmiş bir gizleme tekniği kullanmamıştır (ki API çağrılarının açıkça görünmesi bunu destekler) ya da dosya, FLOSS'un mevcut kurallarıyla tespit edilemeyecek kadar özel (custom) ve karmaşık bir şifreleme algoritması/paketleyici barındırmaktadır. Her halükarda, elde edilen statik API dizileri, bir sonraki aşama olan Dinamik Analiz (Dynamic Analysis) veya Tersine Mühendislik (Reverse Engineering - IDA Pro / Ghidra) için kritik bir yol haritası sunar.

## Kötü Amaçlı Dosyaları İncelemek!

# Statik Analiz Aşaması ve İlk Bulgular

İncelememize konu olan vaka, şüpheli bir `windows.exe` dosyasının 24 Eylül 2024 saat 03:43'te bir kullanıcı tarafından indirilmesiyle başlıyor. Olay müdahale (Incident Response) ekibi tarafından izole edilen bu dosya, analiz için

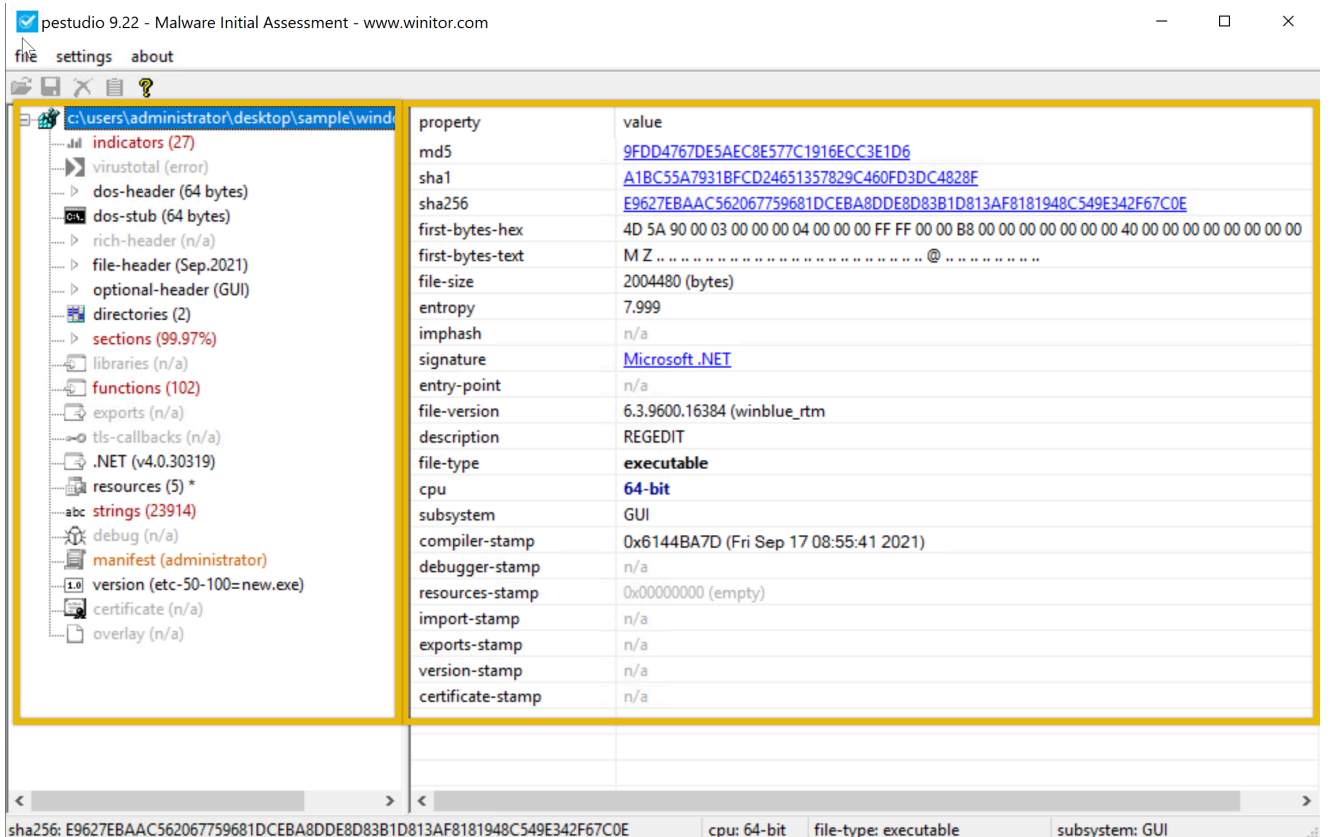
`C:\Users\Administrator\Desktop\Sample` dizinine aktarılmıştır.



Bu noktada ilk hamlemiz zararlı yazılımı çalıştırmadan kod yapısını, import edilen kütüphaneleri ve derleme meta verilerini incelediğimiz **Static Analysis** (Statik Analiz - Zararlı dosyayı çalıştırmadan kapalı kutu yaklaşımıyla dosya imzalarını ve yapısını inceleme tekniği) aşamasıdır.

## PEStudio Üzerinden Yapısal Analiz (Header ve IAT İncelemesi)

Analizimize sektör standartlarından biri olan **PEStudio** ile başlıyoruz. Bu araç, çalıştırılabilir (executable) dosyanın anatomisini ortaya döker. Dosyayı araca yüklediğimizde elde ettiğimiz kritik bulgular ve bunların teknik çevirisi şu şekildedir:





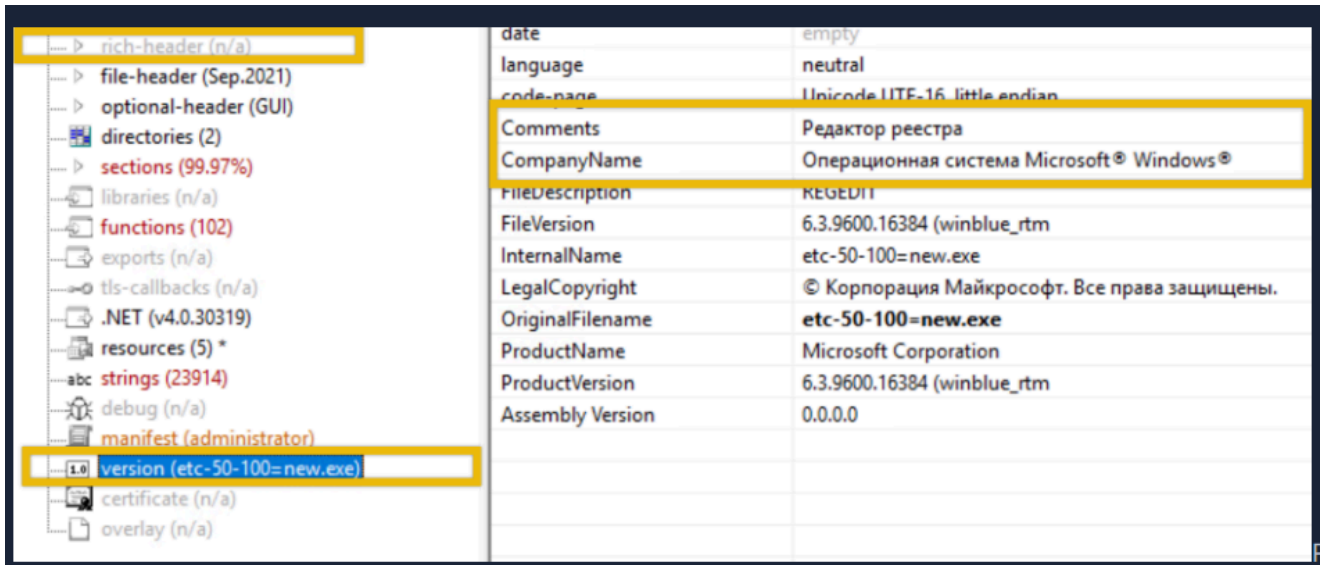
## Hash Değerleri ve İmza Kontrolü: Dosyanın hesaplanan MD5 özeti

9FDD4767DE5AEC8E577C1916ECC3E1D6 ve SHA-1 özeti

A1BC55A7931BFCD24651357829C460FD3DC4828F olarak tespit edilmiştir. **ÖNEMLİ:** Bu hash değerlerinin **VirusTotal** gibi köklü tehdit istihbarat veritabanlarında sorgulanması elzemdir. Eğer herhangi bir antivirüs motoru veya veritabanı bu hash değerini tanıımıyorsa, büyük ihtimalle "Zero-Day" (sıfırıncı gün) veya kurumumuz için özel olarak derlenmiş, henüz keşfedilmemiş taze bir zararlı yazılım kampanyası ile karşı karşıyayız demektir.

**Masquerading (Kimlik Gizleme) Girişimi:** Dosyanın metadata kısmındaki açıklamada kendini Windows Registry Editor (REGEDIT) olarak tanıttığı görülüyor. Meşru bir

regedit.exe dosyasının işletim sistemindeki mutlak konumu C:\Windows\System32 dizinidir. Bu dosyanın kullanıcının indirme dizininde bulunması ve REGEDIT kılığına girmesi, tipik bir savunma atlatma (Evasion) taktiğidir.



**Dil / Bölge Anomalileri:** Dosyanın derleme bilgilerinde ("Редактор реестра" - "Registry Editor", "Операционная система Microsoft® Windows®") Rusça metinler tespit edilmiştir. Eğer organizasyonunuz Rusça konuşulan bir bölgede faaliyet göstermiyorsa, bu durum doğrudan tehdit aktörünün kökenine (Attribution) işaret eden ve kurumsal güvenlik açısından derin çıkarımlar yapmamızı sağlayan kritik bir göstergedir (IOC).

**Rich Header Eksikliği ve Obfuscation:** PEStudio analizinde dosyanın bir **Rich Header** barındırmadığı fark edilmiştir. **DİKKAT:** Microsoft derleyicileri (compilers), derleme ortamı hakkında bilgi veren bir Rich Header'ı PE (Portable Executable) dosyasına otomatik olarak ekler. Bu başlığın eksik olması, dosyanın statik analiz araçlarını kör etmek ve imza tabanlı tespitleri atlatmak amacıyla özel araçlarla **Packed** (sıkıştırılmış) veya **Obfuscated** (kodu karıştırılmış/karmaşılaştırılmış) olduğuna dair kesin bir kanıttır. Bu, sıradan bir script kiddie işi değil, sofistike bir zararlı yazılım davranışdır.

**IAT (Import Address Table) Analizi:** Zararlının işletim sisteminden çağırdığı Windows API fonksiyonlarını listelemek, zararlının çalıştığında "ne yapabileceğini" öngörmemizi sağlar. PEStudio'nun blacklist (kara liste) sekmesi, potansiyel olarak kötüye kullanılan API'leri en üste taşır. Tespit edilen kritik fonksiyonlar şunlardır:

| functions (102)                   | blacklist (1) | functions (102)           | blacklist (1) |
|-----------------------------------|---------------|---------------------------|---------------|
| <u>set WindowStyle</u>            | -             | <u>Process</u>            | -             |
| <u>set CreateNoWindow</u>         | -             | <u>Thread</u>             | -             |
| <u>set UseShellExecute</u>        | x             | <u>Path</u>               | -             |
| <u>set RedirectStandardOutput</u> | -             | <u>Assembly</u>           | -             |
| <u>Start</u>                      | -             | <u>ResourceManager</u>    | -             |
| <u>Sleep</u>                      | -             | <u>File</u>               | -             |
| <u>GetTempPath</u>                | -             | <u>String</u>             | -             |
| <u>Combine</u>                    | -             | <u>Environment</u>        | -             |
| <u>GetExecutingAssembly</u>       | -             | <u>SpecialFolder</u>      | -             |
| <u>GetObject</u>                  | -             | <u>Exception</u>          | -             |
| <u>WriteAllBytes</u>              | -             | <u>MemoryStream</u>       | -             |
| <u>GetEntryAssembly</u>           | -             | <u>CryptoStream</u>       | -             |
| <u>get Location</u>               | -             | <u>RijndaelManaged</u>    | -             |
| <u>Concat</u>                     | -             | <u>SymmetricAlgorithm</u> | -             |
| <u>GetFolderPath</u>              | -             | <u>CipherMode</u>         | -             |
| <u>set WorkingDirectory</u>       | -             | <u>Rfc2898DeriveBytes</u> | -             |
| <u>set KeySize</u>                | -             | <u>DeriveBytes</u>        | -             |
| <u>set Mode</u>                   | -             | <u>ICryptoTransform</u>   | -             |
| <u>GetBytes</u>                   | -             | <u>Stream</u>             | -             |
| <u>CreateDecryptor</u>            | -             | <u>CryptoStreamMode</u>   | -             |
| <u>Write</u>                      | -             | <u>IDisposable</u>        | -             |
| <u>Close</u>                      | -             | <u>&lt;Module&gt;</u>     | -             |
| <u>Dispose</u>                    | -             | <u>UpdateRowErrorText</u> | -             |

- set\_UseShellExecute : Bu fonksiyon, process'in (işlemin) işletim sisteminin shell'ini kullanarak yeni alt işlemler (child processes) başlatmasına olanak tanır. Zararlı yazılımlar genellikle komut satırı veya PowerShell üzerinden asıl **Payload** (zararlı yük) kodunu çalıştırmak için bu API'ye başvurur.





windows.txt - Notepad

File Edit Format View Help

set\_WorkingDirectory

Exception

MemoryStream

CryptoStream

System.Security.Cryptography

RijndaelManaged

SymmetricAlgorithm

set\_KeySize

set\_Mode

CipherMode

GetBytes

Rfc2898DeriveBytes

DeriveBytes

CreateDecryptor

ICryptoTransform

Stream

CryptoStreamMode

Write

Close

IDisposable

Dispose

ToArray

REGEDIT

Microsoft

Windows

Microsoft Corporation

6.3.9600.16384 (winblue\_rtm

\$131e14cc-f38c-4324-a4b4-f9f47d8f4690

WrapNonExceptionThrows

ffff

`ff`

`fof`

ffff

<?xml version="1.0" encoding="utf-8"?>

<assembly manifestVersion="1.0" xmlns="urn

<

- `CryptoStream`, `RijndaelManaged`, `CipherMode`, `CreateDecryptor`: Doğrudan kriptografik operasyonlara, spesifik olarak AES (Rijndael) şifrelemesine işaret eden API çağrılarıdır. Zararlı yazılımın ağ üzerinden komuta kontrol sunucusuyla yapacağı haberleşmeyi (C2 Traffic) şifreleyerek IDS/IPS sistemlerinden kaçırmaya çalıştığını veya hedefin dosyalarını kilitleyecek bir Ransomware (Fidye Yazılımı) rutini barındırdığını kanıtlar.

## FLOSS ile Gelişmiş String (Metin) Analizi

PEStudio bulgularımızı doğrulamak ve sıkıştırılmış metin dizilerini çıkarmak için

`C:\Users\Administrator\Desktop\Sample` dizininde PowerShell üzerinden **FLOSS**

(FireEye Labs Obfuscated String Solver - Klasik "strings" aracının aksine, bellekteki deobfuscation algoritmalarını simüle ederek gizlenmiş metinleri çözen gelişmiş araç) aracını çalıştırıyoruz.

```
PS C:\Users\Administrator\Desktop\Sample > FLOSS.exe .\windows.exe > windows.txt
```

Bu komutta `.\windows.exe` dosyasını analiz etmesi için FLOSS aracına veriyor ve `>` yönlendirme (redirection) operatörü ile ekrana basılacak olan devasa çıktıyı daha rahat analiz edebilmek için aynı dizindeki `windows.txt` isimli dosyaya yazdırıyoruz.

```
WARNING: floss: .NET language-specific string extraction is not supported yet
WARNING: floss: FLOSS does NOT attempt to deobfuscate any strings from .NET binaries
INFO: floss: disabled string deobfuscation
INFO: floss: extracting static strings
INFO: floss: finished execution after 0.34 seconds
INFO: floss: rendering results
```

**Çıktı Analizi ve Mantıksal Çıkarım:** Terminal loglarını okuduğumuzda FLOSS'un .NET uyarısı verdiğini görüyoruz. Bu hata mesajı son derece değerlidir; bize zararlı yazılımın C# veya VB.NET dillerinden biriyle (.NET Framework kullanılarak) yazıldığını söyler. FLOSS,

.NET tabanlı karmaşıklık algoritmalarını henüz deobfuscate edemediği için bu modülü devre dışı bırakmış ("disabled string deobfuscation"), ancak yine de statik metin dizilerini ("extracting static strings") başarıyla çekmiştir. windows.txt dosyasını açıp incelediğimizde, PEStudio'da gördüğümüz kriptografik API çağrılarının ve shell execution metinlerinin aynen burada da yer aldığını görüyoruz. İki farklı araç aynı sonucu verdiğinden, bulgularımızın doğruluğunu kesinleştirmiş oluyoruz.

## Dinamik Ağ Analizi: C2 (Command & Control) Tespiti

Statik analizi bitirdikten sonra ortamımızı hazırlıyoruz ve aynı dizinde yer alan

C:\Users\Administrator\Desktop\Sample\cobaltstrike.exe isimli yeni nesil bir tehdit dosyasının ağ davranışlarını yakalamak üzere **Dynamic Analysis** (Dinamik Analiz - Zararlıyı kontrollü ortamda çalıştırıp sistemdeki ve ağdaki anlık davranışlarını izleme) aşamasına geçiyoruz. Amacımız, dosyanın bir **C2** (Command and Control - Saldırganın zararlı yazılımı yönettiği uzak sunucu) ile iletişim kurup kurmadığını kanıtlamaktır.

## Process Explorer ile Süreç Hiyerarşisi İncelemesi

Dosyaya çift tıklayarak çalıştırıyor ve sistemdeki anlık durumları gösteren **Process Explorer** aracını açıyoruz. GUI üzerinden çalıştırdığımız için işletim sistemi mantığı gereği bu eylemin **Parent process'i** (Ana/Ebeveyn işlem) Explorer.exe olacaktır. Gerçekten de hiyerarşiye baktığımızda cobaltstrike.exe 'nin Explorer.exe altında bir **Child process** (Alt işlem) olarak türediğini doğruluyoruz.

|                  |        |          |           |                                   |                              |
|------------------|--------|----------|-----------|-----------------------------------|------------------------------|
| explorer.exe     | < 0.01 | 55,164 K | 195,688 K | 1208 Windows Explorer             | Microsoft Corporation        |
| cobaltstrike.exe |        | 1,984 K  | 9,152 K   | 4756                              |                              |
| procexp64.exe    | 0.76   | 42,956 K | 70,824 K  | 396 Sysinternals Process Explorer | Sysinternals - www.sysint... |
| sihost32.exe     |        | 37,648 K | 40,100 K  | 4680 Shell Infrastructure Host    |                              |

Bu process'in o anki işletim sistemi tarafından atanan Process ID'si (PID) **4756** olarak saptanmıştır. Bu değere sağ tıklayıp Properties (Özellikler) menüsünden **TCP/IP** sekmesine gittiğimizde, zararlının anlık olarak kurduğu veya dinlediği (LISTENING / ESTABLISHED) ağ bağlantılarının hedef (Destination) adreslerini görebiliriz.

cobaltstrike.exe:4756 Properties

Image

Performance

Performance Graph

Disk and Network

GPU Graph

Threads

TCP/IP

Security

Environment

Job

Strings

☒ Resolve addresses

Pr...

Local Address

Remote Address

State^

TCP win-7bs0gh8qp51.eu-west-1.compute.internal:49956

47.120.46.210:hosts2-ns

SYN\_SENT

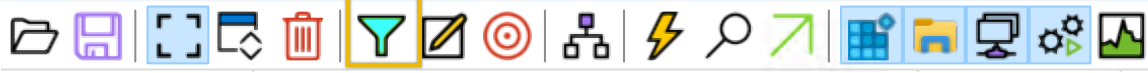
## Process Monitor (Procmon) ile Tehdit Doğrulaması (Verification)

Siber güvenlik analitiğinde tek bir aracın çıktısına asla %100 güvenilmez; bulgular çapraz doğrulamaya (cross-verification) tabi tutulmalıdır. Process Explorer'da tespit ettiğimiz ağ bağlantısını, işletim sistemindeki dosya, kayıt defteri ve ağ aktivitelerini anlık kaydeden çok daha detaylı bir araç olan **Process Monitor (Procmon)** ile teyit edeceğiz.

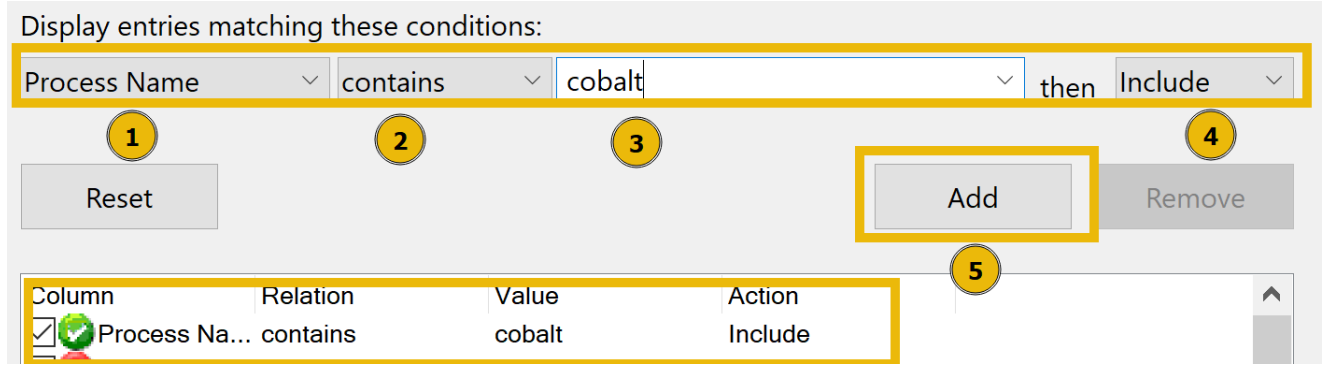
Process'i sonlandırıp yeniden başlatmadan önce Procmon'u açıyoruz. Procmon saniyede on binlerce Windows event'i yakaladığı için samanlıkta iğne aramamak adına **Filtreleme (CTRL + L)** özelliğini devreye sokuyoruz:

## Process Monitor - Sysinternals: [www.sysinternals.com](http://www.sysinternals.com)

File Edit **Event** Filter Tools Options Help



1. İlk sütunda kural türü olarak **Process Name** seçilir.
2. İlişki türü olarak **contains** (içerir) belirlenir.
3. Değer alanına aranacak spesifik kelime olan **cobalt** yazılır.
4. Aksiyon olarak **include** (sadece bu kurala uyanları dahil et) seçilir.
5. **Add** butonuna basılarak kural listeye eklenir ve **Apply** ile filtre uygulanır.



Bu ince filtrenin ardından Procmon ekranı sadeleşir ve sadece **cobaltstrike.exe** process'ine ait davranışlar ekranda listelenir. Ağ (Network) operasyonlarına ait satırları incelediğimizde, zararlının **47.120.46.210** IP adresine dışarı doğru (outbound) bir bağlantı kurduğu kesin olarak kanıtlanmıştır.

| Time o...  | Process Name     | PID  | Operation      | Path                                                                        | Result  |
|------------|------------------|------|----------------|-----------------------------------------------------------------------------|---------|
| 8:29:25... | cobaltstrike.exe | 1120 | TCP Reconnect  | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49961 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:29:38... | cobaltstrike.exe | 1120 | TCP Disconnect | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49961 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:29:41... | cobaltstrike.exe | 1120 | TCP Reconnect  | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49962 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:29:47... | cobaltstrike.exe | 1120 | TCP Reconnect  | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49962 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:29:59... | cobaltstrike.exe | 1120 | TCP Disconnect | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49962 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:30:02... | cobaltstrike.exe | 1120 | TCP Reconnect  | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49963 -> 47.120.46.210:hosts2-ns | SUCCESS |
| 8:30:08... | cobaltstrike.exe | 1120 | TCP Reconnect  | WIN-7BS0GH8QP51.eu-west-1.compute.internal:49963 -> 47.120.46.210:hosts2-ns | SUCCESS |

**Sonuç:** Bu IP adresi, zararlının veri çaldığı (Exfiltration) veya saldırgandan yeni komutlar aldığı aktif C2 sunucusudur ve acilen güvenlik duvarlarında (Firewall/EDR) engellenmesi (Block/Drop) gereken öncelikli hedeftir.